INDEX

NAME: K. Arafath STD.: _____ SEC.: _____ ROLL NO.: 161 SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	16/7/26	Basic Networking commands in windows & Linux		Bll 6/8
2.	23/7/26	Study of different types of network cables		Bll 6/8
3.1	23/7/26	HUB		Bll 6/8
3.2	23/7/26	SWITCH		
3.3	23/7/26	HUB & SWITCH		
4.	30/7/26	Setup & Configure LAN using switch or ethernet		Bll 6/8
5.	1/8/26	Packet capture tool		Bll 6/8
6.	6/8/26	Hamming code error correction		Bll 20/8
7.	6/8/26	Sliding window protocol		
9.	10/8/26	Cisco Packet Tracker		Bll 16/10
10.	10/9/26	Networking using layer		Bll 16/10
12.	17/9/26	Network using Router		Bll 16/10
13.	24/9/26	Routing at Network layer		Bll 16/10
13	8/10/26	Implement your own Ping Program		Bll 16/10
14	10/10/26	Write a code using raw sockets to implement Packet Sniffing		Bll 16/10
8.	8/10/26	Wireless LAN		Bll 16/10
15.	8/10/26	Analyse Various Types of server using Webalizer tool.		Bll 16/10.

GKC0925203 B

Exp: 1
16/7/25

Basic Network Commands

Aim:

To study and use basic networking commands for monitoring, trouble shooting and verifying network connectivity

Commands:

1) Ping:

purpose: Checks whether a remote host is reachable and measures response time

Sample command: Ping google.com

Sample Output: Reply from 142.250.180.14:
byte=32 time=18ms TTL=114

2.) Ipconfig:

purpose: Display IP address, subnet mask, and default gateway

Sample command: ipconfig

Sample output: IPV4 Address: 192.68.1.10

3.) Ip config:

purpose: Show IP address and interface

Sample command: ipconfig

Sample Output: inet 192.168.1.20 netmask
255.255.255.0

4) Tracert (windows):

purpose: Identifies each router (hop) between source and destination

Sample command: tracert google.com

Output: 1 192.168.1.1
2 10.0.0.1

5) Traceroute (Linux):
Purpose: Identifies each route (hop) between source and destination
Sample command: tracert google.com
Output: 1. 192.168.1.1
2. 10.0.0.1
3. 142.250.179.14

6) nslookup:
purpose: Resolves a domain name into address
Sample command: ns lookup google.com
Output: Name: google.com
address: 142.250.183.14

7) Netstat:
purpose: Shows active connections, listening port and routing info
command: netstat -a
Output: TCP 0.0.0.0:80 listening

8.) arp:
purpose: Maps Ipaddress to MAC address
command: arp -a
Output: 192.167.1.1.00-1a-2b-3c-4d-5e
dynamic

9.) hostname:
purpose: Shows the System network hostname
Command: hostname
Output: Desktop-ABC123

10.) getmac :

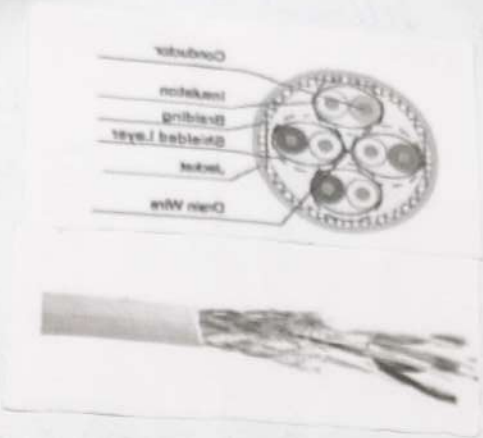
purpose : Shows the physical address of network adapter

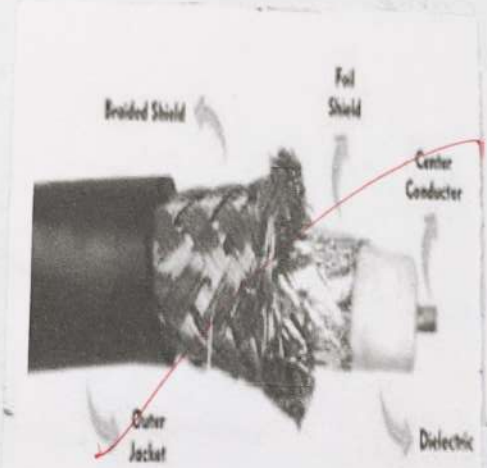
Command : getmac

Output : 00-1A-2B-3C-4D-5E

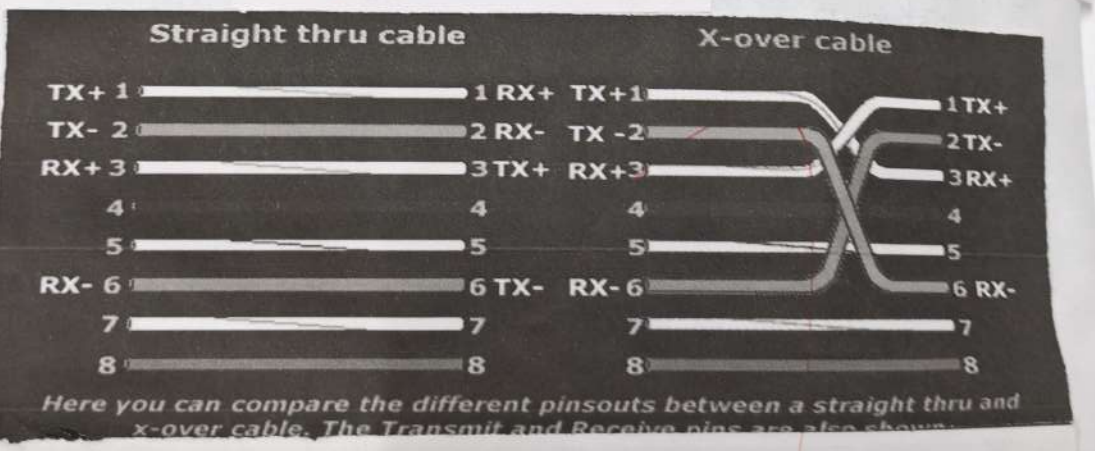
Result :

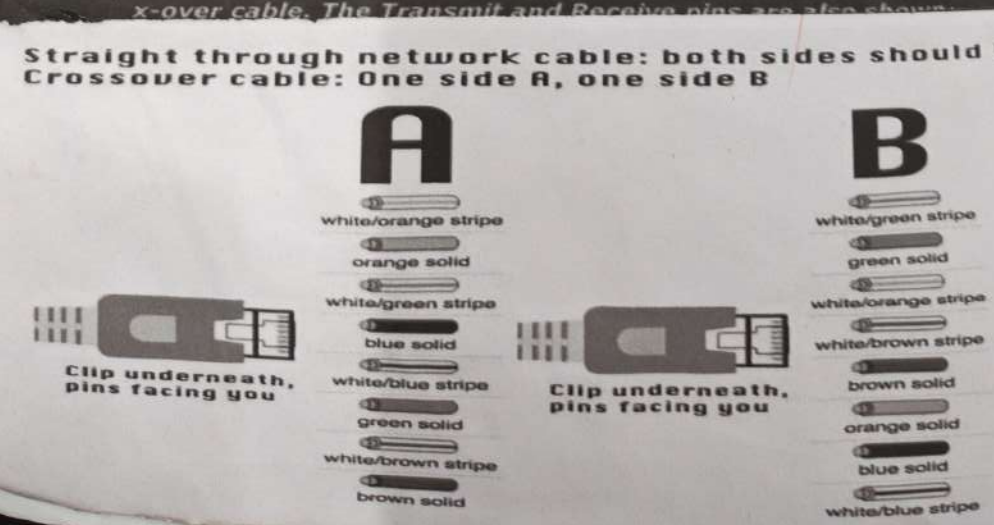
The commands are executed successfully and output is obtained











Coaxial Cable
RG-6
RG-59
RG-11

Fibre Optic Cable
Single mode
multi mo

Make
Tools a
E
gigabet
well,
A
netwo
in th

Step

Step

Exp: 2
23/7/26

Study of different types of network cables

Aim:

To study and understand different types of network cables. To make your own ethernet cross over / straight cable

a) Understand different types of network cables

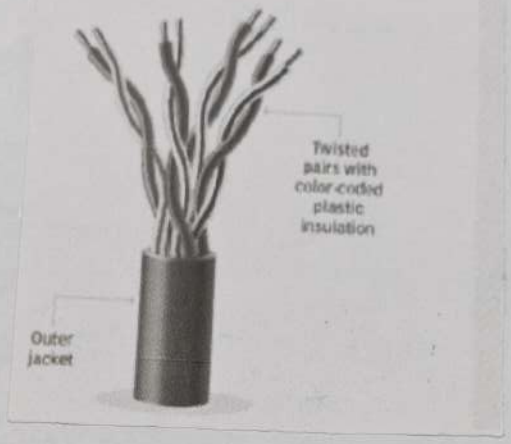
1. Unshielded Twist Pair (UTP) cable
2. Shielded Twist Pair (STP) cable
3. Coaxial cable
4. Fabric optic cable

cable Type	category	Maximum Data Transitions	Advantages/ Disadvantages	Application / use
UTP	category 3	10 kps	Advantages: cheaper in cost Easy to install as they have a smaller overall diameter Disadvantages: More prone to electromagnetic Inference & noise	10 box - T Ethernet
	category 5	upto 100 mbps		Fast Ethernet Gigabit Ethernet
	category 5e	10 bps		Fast ethernet Gigabit ethernet
STP	Category 6, 6a	10 Gbps	Advantage: Shielded Faster than UTP	Gigabit ethernet 10G ethernet widely used in data centres
	Category 7	10 Gps	less susceptible to noise and interfer Disadvantage: Expensive Greater install effort	Gigabit Ethernet 100 Ethernet

Exp:-2
23/7/26

Aim:-
To
types o
ethernet
a) Un
1. Un
2. S
3. C
4. F

cable Type	
UTP	
STP	





Shielded twisted pair

Step 3: After you will need to angle the wires there should be four referencing back to the sheet

Step 4: Once the order is correct, brush them together in a line, and if there are any that stick at further than others. To do so, hold the plug with the clip side facing away from you

Step 5: Next, push the cable right in. The notch at the end of plug needs to be just over the cable shield

Step 6: After the wires are securely sitting inside the crimping tool and push down. It should be shaped correctly, but pushing too hard.

Step 7: Lastly, repeat the other end using diagram B/ using diagram A

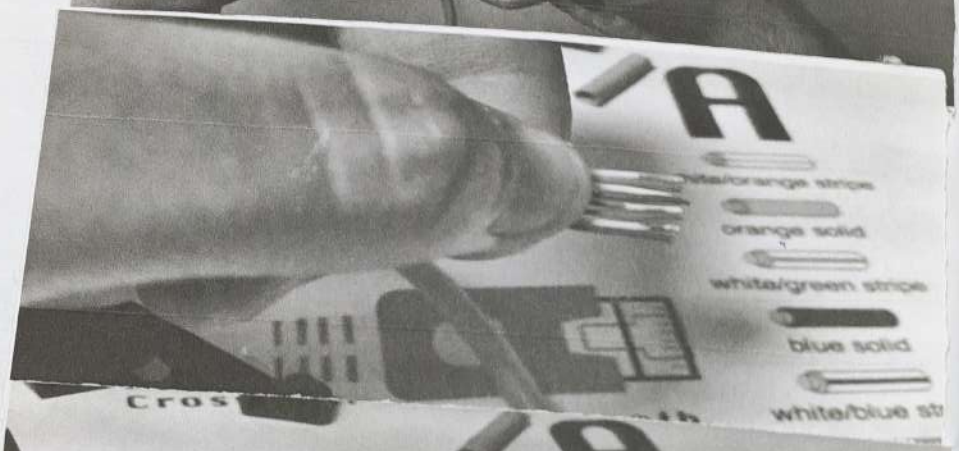
Result:

Thus, different types of network cables were studied carefully and straight through and gas over ethernet cables were prepared and tested successfully.

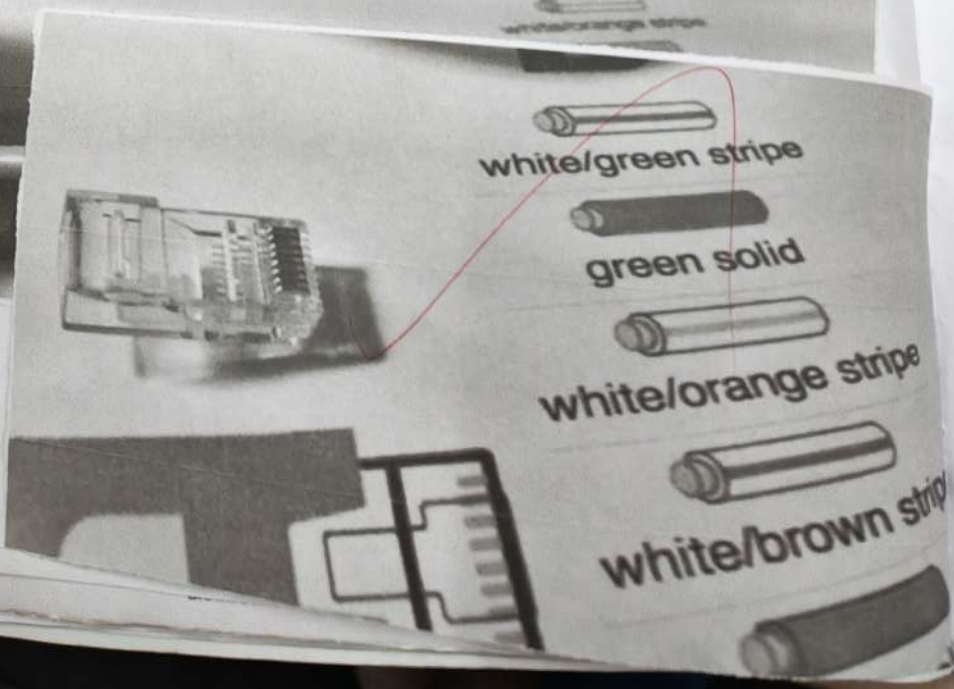
90











Step 3: After yo
wires th
referen

Step 4: Once the
together
any th
others
the cli

Step 5: Next,
notch
be in

Step 6: After
inside
It sl
but

Step 7: Las
dia

Result:
Thus
cables we
straight
cables
successfu

90

Coxial Cable	RG-6 RG-39 RG-11	10-100 Mbps	Advantages: High bandwidth Immune to fore low loss bandwith Disadvantage: Limited distance Cost Size is bulky	Speed of signal is 200m
Fibre Optic Cable	Single mode multi mode	100 Gbps	Advantages: High speed High bandwidth High security Disadvantages Expensive Required skills installers	Maximum distance of fibre optics cable is around 10m

Make your own ethernet cross over cable:

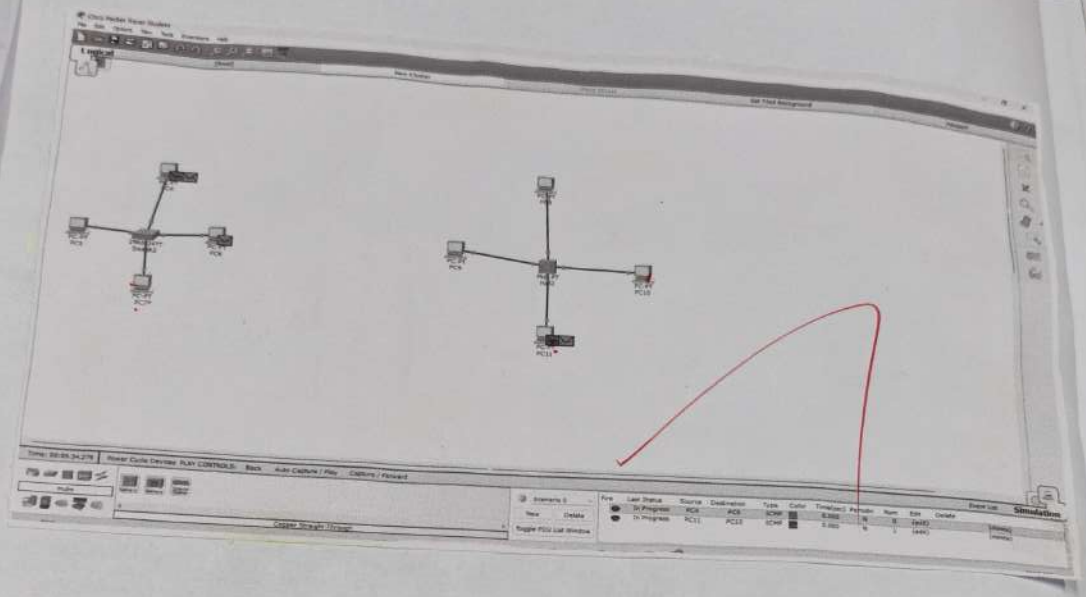
Tools and Parts needed:

Ethernet cabling CAT5E is orfied for gigabet support, but CAT5 cabling works as well, just over shorter distance.

A ginping tool, this is an all in one networking tool shaped to push down the pins in the plug and strip and sheilding off

Step 1: To start construction of the device - begin by threading shields into cable

Step 2: Next, strip apprx 1.5cm of cable shielding from both ends.



: 3.1
7/26

Aim:
To study
how it forwa

Steps:
1) Open ci
2) Drag a
3) Conne
straight
4) confi
5) Sele
common
6) Se
7) Su
how c

Res

to
th
a

Result:

This Cisco packet tracer environment is successfully explored and a simple network was designed. The interface components, device connections, IP configuration and packet transmissions were successfully understood using realtime and simulation mode.

p. 3.1

3/10/26

Understanding the cisco packet tracer environment

Aim:

To understand the cisco packet tracer environment and during a simple network using its interface and networking components

Steps:

1) Open the cisco packet tracer application
2) Observe the user interface, including the menu bar, main tool bar, workspace, common tool
3) From the network component box, select the required network devices such as hubs, git and switch.
4) Drag and drop the devices into workspace
5) connect the devices using copper straight through cables
6) configure IP address for the PCs using the desktop → IP configuration option
7) verify that all links are active
8) use the add simple PDU tool to send a packet between two PCs.
9) Observe packet transmission in realtime mode and simulation mode
10) Save the network topology.

Exp: 3.2
23/9/26

SWITCH

Aim:

To study the behaviour of a switch by observing how it forwards data packets using cisco packet tracer

Steps:

1) Open cisco packet tracer
2) Drag and place 1 switch and 4 generic PCs
3) Connect to all PCs to the switch using copper straight through cable
4) Configure IP address for all PCs in same network.
5) Select and PDU from common tool bar
6) Send a packet from one PC to another
7) Switch to simulation mode and observe how the switch forward the packet

Result:

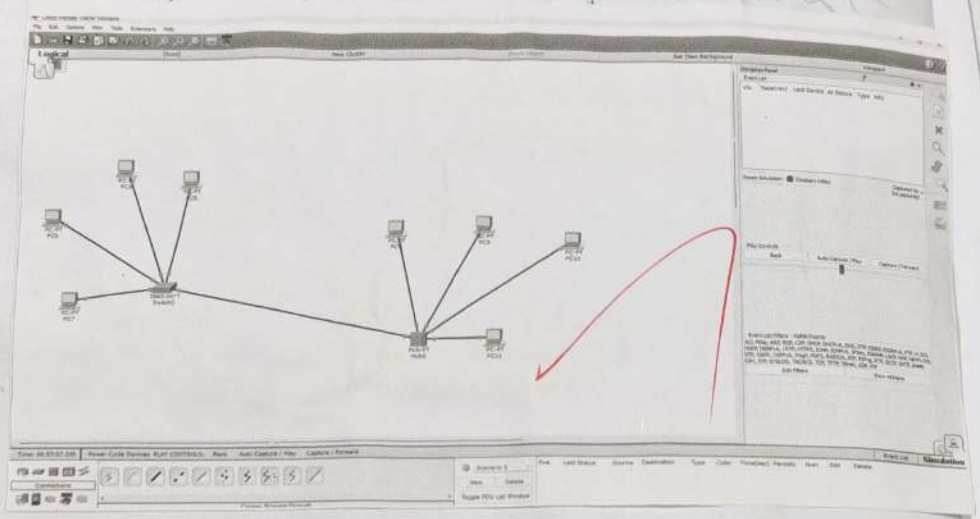
The switch forward the packet only to the destination PC by using its MAC address the table. This reduce unnecessary network tropic and improves communication efficiency.

p: 3.2
3/1/26

SWI

Aim:
To study the
observing how
using cisco pack

Steps:
1) Open cisco
2) Drag and
3) Connect
copper straight
4) Configure
same network
5) Select a
6) Send a
7) Switch
how the



Result:
The
to the d
the tab
network
efficie

Exp: 3.1
3/1/26

HUB

Aim:

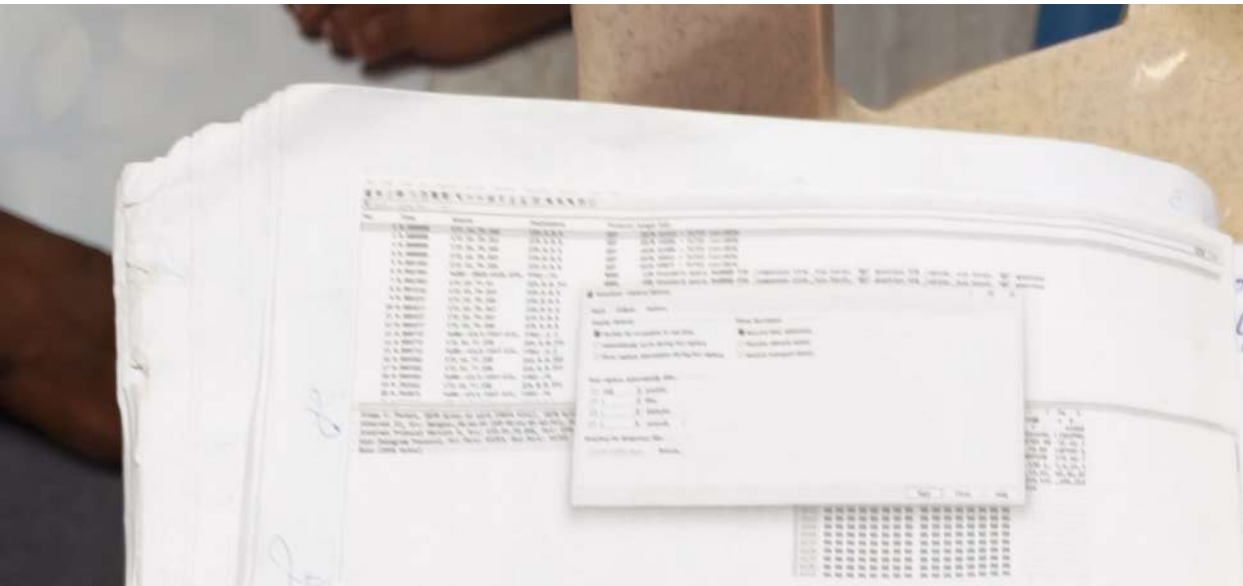
To study the behaviour of a hub by observing how it forwards data packets using cisco packet

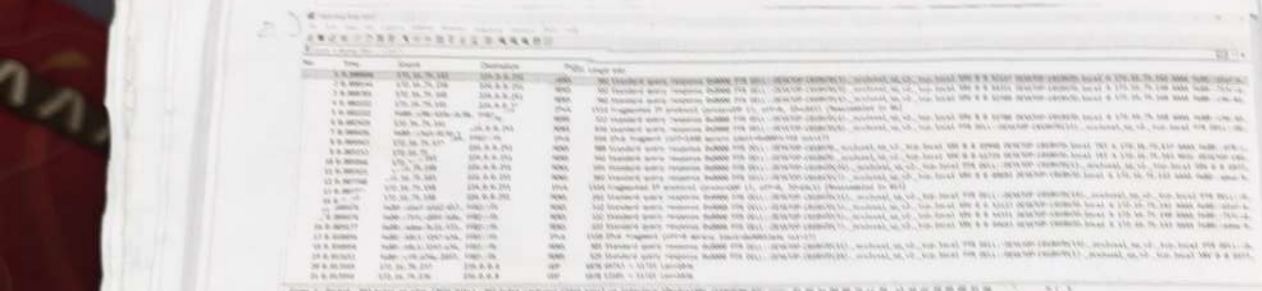
Steps:

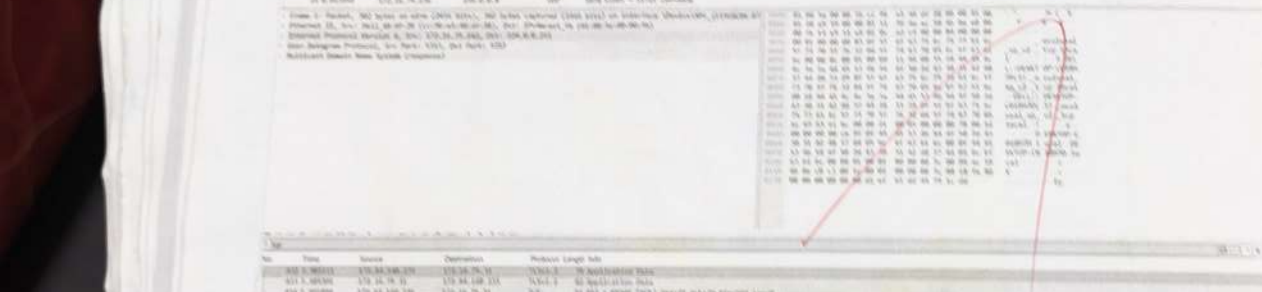
1) Open cisco packet tracer
2) Drag and place 1 hub and 4 generic PCs
3) Connect all PCs to the hub using copper straight through cable.
4) Configure the IP address for all PCs same
5) Select Add symbol PCU from the common tool
6) Send a packet from one PC to other
7) Switch to stimulation mode and observe how does forward packet

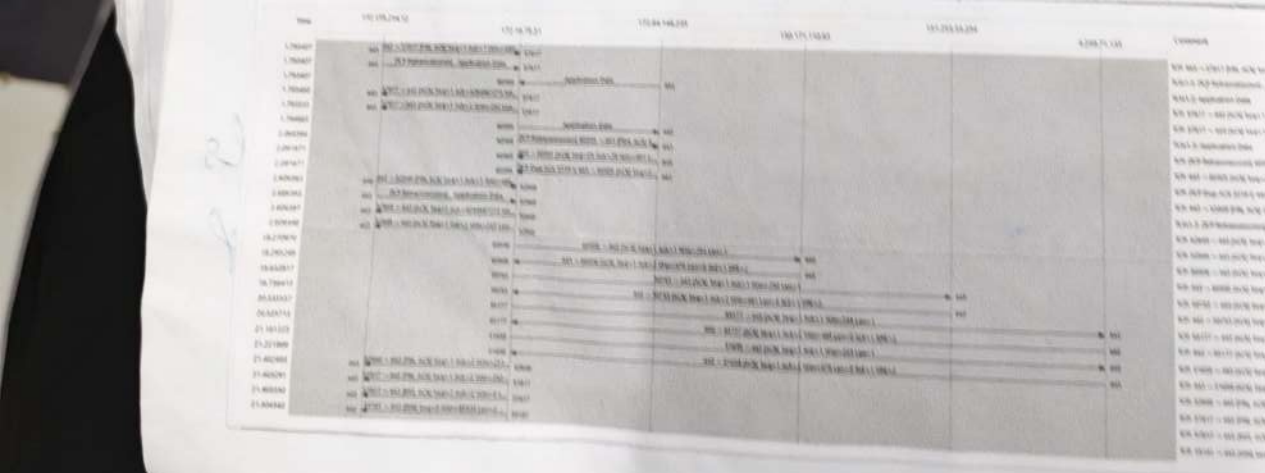
Result:

The hub broadcast the received packet to all connected devices. Every PC receives the packet, but only the destination PC accept its while the others discards it









Experiment on

Aim:
To study the
packet capture
and to capture
at different

Procedure:
1) Open the
2) From the
select local
3) Go to
local area
4) Set
Set the cap
packets.
5) click
6) Gra
opening a
7) Ob
8) Se
9) O
of the
10)
filter

Result
Succe
Netw
fiel
flo

Exp: 3.3
23/4/26

HUB AND SWITCH

Aim:

To compare the packet forwarding behaviour of a hub and a switch using cisco packet tracer

Steps:

1) Create one network using a hub and another using a switch, each connected to 4PC

2) Configure IP address for all PCs in both networks

3) Send a simple PDU between two PCs in each network

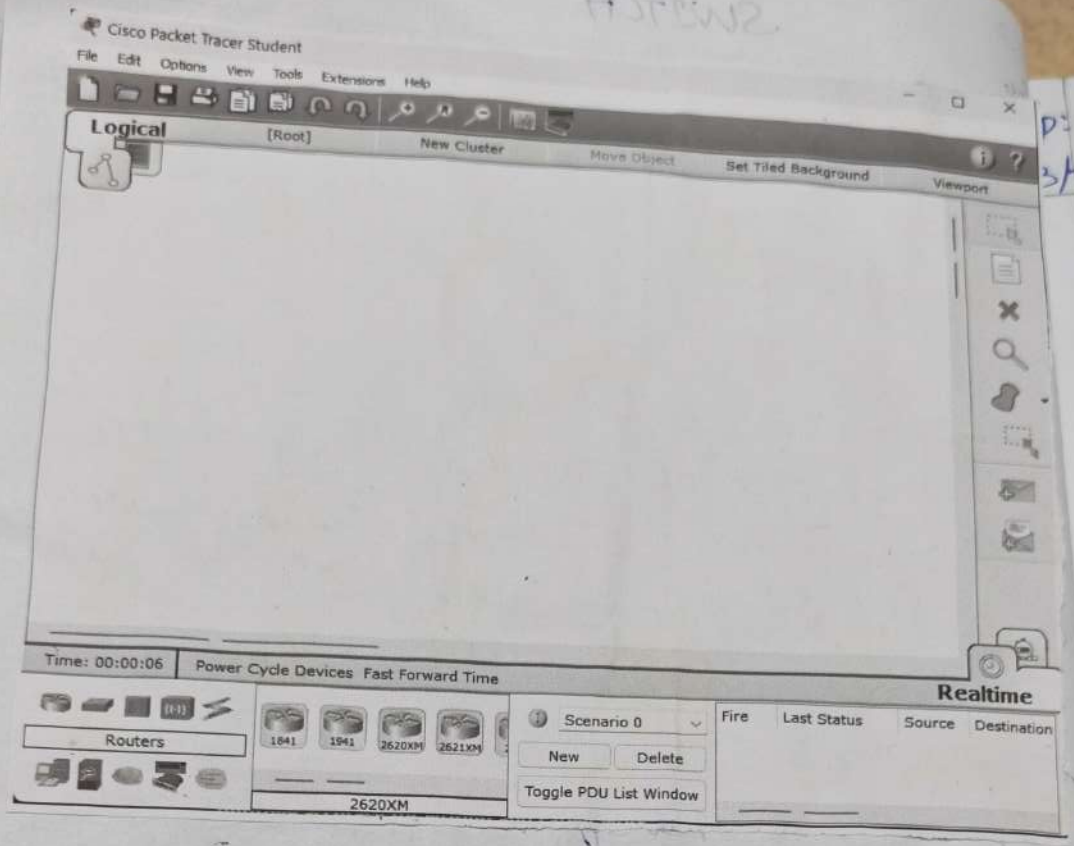
4) Observe packet transmission in simulation.

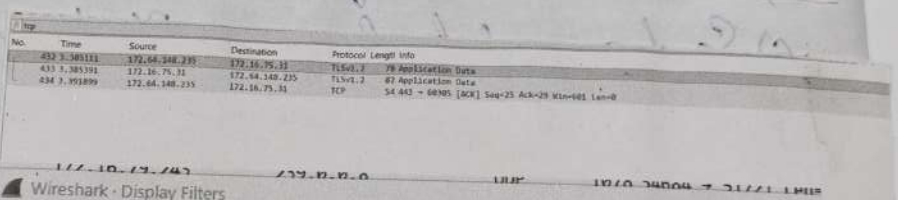
5) Compare how the hub and switch forward packets

Result:

The hub broadcast packet to all connected devices, whereas the switch forwards packets only to the intended destination. Therefore the switch provids better performance, reduced networks and improved efficiency compared to hub.

8/5





Wireshark · Display Filters

Filter Name	Filter Expression
Ethernet address 00:00:5e:00:53:00	eth.addr == 00:00:5e:00:53:00
Ethernet type 0x0806 (ARP)	eth.type == 0x0806
Ethernet broadcast	eth.addr == ff:ff:ff:ff:ff:ff
No ARP	not arp
IPv4 only	ip
IPv4 address 192.0.2.1	ip.addr == 192.0.2.1
IPv4 address isn't 192.0.2.1	ip.addr != 192.0.2.1
IPv6 only	ipv6
IPv6 address 2001:db8::1	ipv6.addr == 2001:db8::1
TCP only	tcp
UDP only	udp
Non-DNS port	!(udp.port == 53 || tcp.port == 53)
TCP or UDP port is 80 (HTTP)	tcp.port == 80 || udp.port == 80
HTTP	http
No ARP and no DNS	not arp and not dns
Non-HTTP and non-SMTP to/from 192.0.2.1	ip.addr == 192.0.2.1 and tcp.port not in {80, 25}

OK Cancel Help

D: 3.3
3/1/26

HUB

Aim:
To comp
behaviour
cisco pack

Steps:
1) Cr
another
2) Con
both
3) S
in each
4)
Stimu
5)
form

Resu

85 Conn
for
BM 6/2 Des
be
in

Ex: 5
12/26

Packet capture Tool

Aim:

To capture, analyze and inspect network packet using wireshark, and understand different network protocols and packet details

Steps:

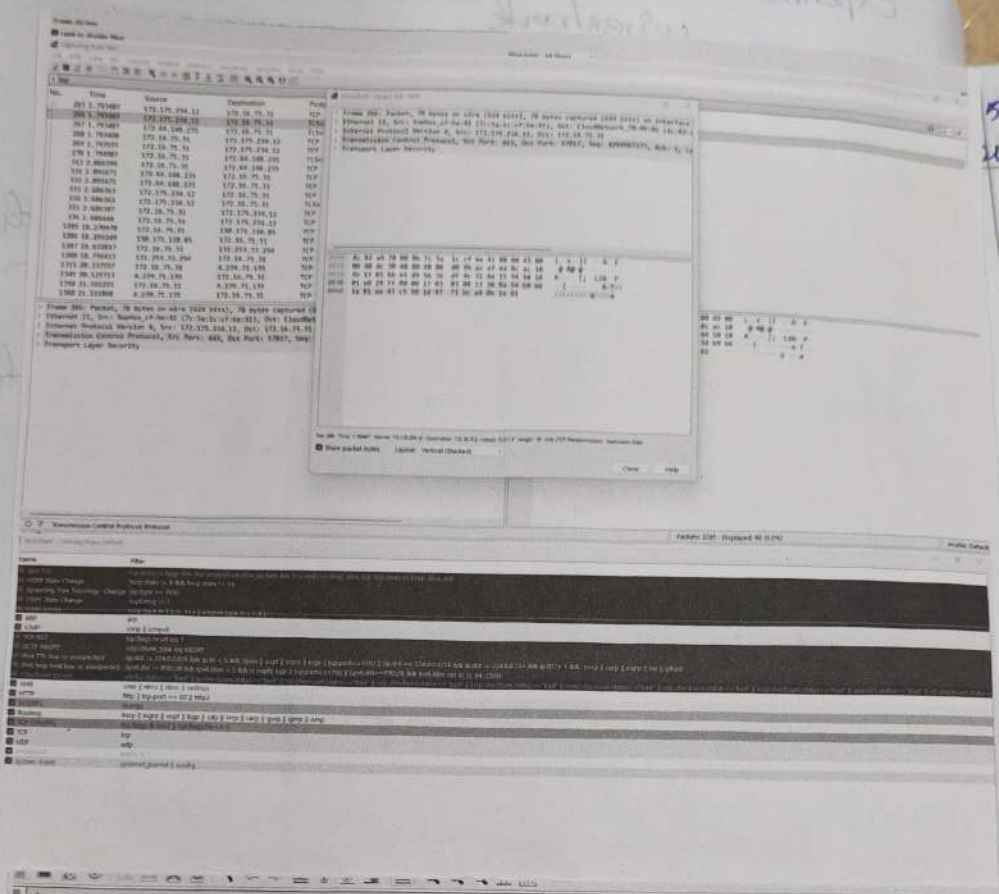
1) Install and open wireshark
2) Select the active network interface
3) Click the start capture button
4) Browse a website or generate network
5) Observe the captured packet in packet list
6) Click any packet to view its packet details and packet bytes
7) Apply display filters to view specific packet
8) Stop packet capture and save the capture file

Result:

Wireshark succesfully captured and analyzed network packet. The captured traffic was filtered and inspected to understand network communication & protocol behaviour.

85

Bully



dns

No.	Time	Source	Destination	Protocol	Lengtl	Info
5047	64.823837	172.16.75.31	172.16.72.1	DNS	70	Standard query 0xf2bf HTTPS dns.google
5048	64.823968	172.16.75.31	172.16.72.1	DNS	70	Standard query 0x2c40 A dns.google
5092	64.872708	172.16.72.1	172.16.75.31	DNS	102	Standard query response 0x2c40 A dns.go
5093	64.872708	172.16.72.1	172.16.75.31	DNS	70	Standard query response 0xf2bf HTTPS dn
5907	73.507194	172.16.75.31	172.16.72.1	DNS	70	Standard query 0xdd84 HTTPS dns.google
5908	73.507610	172.16.75.31	172.16.72.1	DNS	70	Standard query 0x7b95 A dns.google
5909	73.522819	172.16.72.1	172.16.75.31	DNS	102	Standard query response 0x7b95 A dns.go
5910	73.522819	172.16.72.1	172.16.75.31	DNS	70	Standard query response 0xdd84 HTTPS dn
6172	75.347431	172.16.75.31	8.8.8.8	DNS	70	Standard query 0x7402 HTTPS dns.google
6173	75.347571	172.16.75.31	8.8.8.8	DNS	70	Standard query 0xddfb A dns.google
6174	75.377779	8.8.8.8	172.16.75.31	DNS	102	Standard query response 0xddfb A dns.go
6180	75.398557	8.8.8.8	172.16.75.31	DNS	146	Standard query response 0x7402 HTTPS dn

```
> Frame 5047: Packet, 70 bytes on wire (560 bits), 70 bytes
> Ethernet II, Src: CloudNetwork_78:00:0b (4c:82:a9:78:00:
> Internet Protocol Version 4, Src: 172.16.75.31, Dst: 172.
> User Datagram Protocol, Src Port: 53918, Dst Port: 53
> Domain Name System (query)
```

```
0000  7c 5a 1c cf be 41 4c 82  a9 78 00 0b 08 00 45 00  |
0010  00 38 65 85 00 00 80 11  e9 ee ac 10 4b 1f ac 10  .
0020  48 01 d2 9e 00 35 00 24  99 73 f2 bf 01 00 00 01  H
0030  00 00 00 00 00 00 03 64  6e 73 06 67 6f 6f 67 6c  .
0040  65 00 00 41 00 01                                e
```

arp

No.	Time	Source	Destination	Protocol	Lengtl	Info
1343	10.957901	CloudNetwork_78:90:...	Broadcast	ARP	42	Who has 172.16.75.125? Tell 172.16.75.(
1977	14.358615	CloudNetwork_78:90:...	Broadcast	ARP	42	Who has 172.16.77.18? Tell 172.16.75.61
2098	15.471882	CloudNetwork_78:90:...	Broadcast	ARP	42	Who has 172.16.77.18? Tell 172.16.75.61
2400	17.952584	CloudNetwork_78:90:...	Broadcast	ARP	42	Who has 172.16.78.5? Tell 172.16.75.61
2747	24.883561	CloudNetwork_78:90:...	Broadcast	ARP	42	Who has 172.16.75.204? Tell 172.16.75.(
4906	61.332363	CloudNetwork_77:ff:...	Broadcast	ARP	42	Who has 172.16.77.93? Tell 172.16.75.51

5
26 Packet

Aim:
To capture, a
packet using w
different netw

Steps:
1) Install
2) Select th
3) Click th
4) Browse
5) Observe
6) Click
details an
7) Appl
8) Step
capture

Result
u
analy
traff
und
pro

85

Exp: 4 Experiment on Packet capture tool
30/4/26 wireshark

Aim:
To study the features of wireshark as a packet capture and network analysis tool, and to capture, and analyse network packets at different protocol layers.

Procedure:
1) Open the wireshark application on the computer
2) From the available network interfaces, select local area connection / ethernet
3) Go to capture → options and select local area connection
4) Set the required network interface. Set the capture to stop automatically after packets.
5) click start to begin capture network
6) Generate some network activity by opening a website
7) Observe the captured packets in the list
8) Select any packet from the network
9) Observe the hexadecimal representation of the selected packet in bytes plane
10) Apply suitable display filter in the filter bar to analyze specific protocols

Result:
The wireshark packet capture tool was successfully used to capture and analyze network traffic. The packet details protocol fields, packets byte and communication flow was successfully observed.

BULB

8) Extract the original 4 data bits from the corrected 7-bit code word.
9) Repeat step 3 to 8 for all n blocks
10) Stop the program.

Result:

The program to implement error detection and correction using hamming code concept was successfully written in python, executed and verified using a sample input data stream.

Ex: 6
8/26

Error Detection and correction using Hamming code

Aim:

To write a program to implement error detection and correction using Hamming code

Steps:

1) Start the program
2) Read the number of 4-bit data blocks forming the input data stream.
3) For each 4-bit data block $D = d_1\ d_2\ d_3\ d_4$ perform hamming encoding:
   a) Arrange 7 bits with parity bits at position 1, 2 and 4 and data bits at position 3, 5, 6 and 7.
   b) Compute parity using even parity:
   $P1 = d1$ XOR $d2$ XOR $d4$ (cover bit positions 1, 3, 5, 7)
   $P2 = d1$ XOR $d3$ XOR $d4$ (cover bit positions 2, 3, 6, 7)
   $P4 = d2$ XOR $d3$ XOR $d4$ (cover bit positions 4, 5, 6, 7)
   c) Form the 7-bit hamming code word:

P1 P2 d1 P4 d2 d3 d4

4) Stimulate the transmission by introducing a single-bit error at a chosen bit position
5) At the receiver, recompute the check bits (Syndrome) from the received code word:

$C1 = P1$ XOR $d1$ XOR $d2$ XOR $d4$

$C2 = P2$ XOR $d1$ XOR $d3$ XOR $d4$

6) Compute error position
7) If error position = 0, no error has occured.

Program:

```
def encode(d):
h = [0] * 7
h[2] = d[0]
h[4] = d[1]
h[5] = d[2]
h[6] = d[3]
return h
def detect_and_correct(h):
c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
c2 = h[1] ^ h[3] ^ h[5] ^ h[6]
error_pos = c1 * 1 + c2 * 2 + c4 * 4
if error_pos != 0:
print(f"Error detected at bit position : {error
h[error_pos - 1] ^= 1 connecting..")
else:
print("No error detected")
return error_pos
def to_str(bits):
return " ".join(str(b) for b in bits)
def main():
print("=== Hamming code - Error correction ==
h = encode(d)
print(f" original data bits : {to_str(d)}")
print(f" encoded hammingcode : {to_str(h)}")
pos = int(input("Enter bit position to introd
Error (1-7, 0 = no error):
if 1 <= pos <= 7:
h[pos - 1] ^= 1
print(f" Transmitted code (with error
introduced at pos{pos}) : {to_str(h)}")
else:
Print(f" Transmitted code (no error):
```

Error

Aim:

To writ
detection

Steps:

1) Star
2) Read
forming
3) for
perform
a) A
position
3, 5, 6

b)

P1 =
P2
P4
c
P1

a

bits

```
def main():
    print("===SLIDING WINDOW PROTOCOL - FLOW
          CONTROL SIMULATION (Go-back N)
          ===\n")
    total_frames = int(input("enter total no. of
                   frames to be send: "))
    window_size = int(input("enter sender's
                  window size : "))
    if lost_frame == -1:
        lost_frame = None
    Simulate(total_frames, window_size, lost_fram
```

5) Repeat
have been
6) Stop the

Result

85

Exp: 7
6/8/20

Flow control at Datalink layer Using Sliding window protocol

Aim:

To write a program to implement flow control at data link layer using the sliding window protocol and to simulate the flow of frames from one node to another.

Algorithm:

1) Start the program
2) Read the total no. of frames to be sent
3) Initialize base=0 (Seq number of oldest unacknowledged frame) and nextseq=0 (Sequence number of next frame to be sent)
4) Repeat while base < N:
   a) Determine the current sender window as the range
   b) Send all the frame with sequence number from nextseq up to (base+w-1), incrementing nextseq after each frame, since the window restricts the sender to at most w at time.
   c) If the Lost frame lies within frame
   (i) The receiver discards the lost frame and any subsequent out-of-order frames
   (ii) The sender times out retransmit all frames starting from the lost frame
   d) Else (no loss in this batch):
   (i) The receiver accepts each frame in order and sends a cumulative acknowledgement
   (ii) On receiving the ACK for the last frame sent, the sender slides its window forward.

Program:

```
def Simulate (total_frames, window_size, lost_frame):
    base = 0
    next_seq = 0
while base < total_frames:
    window_end = min (base + window_size - 1, tot_frames - 1)
    print ("\n ---------- ")
    print (f" Sender Window : [Frame {base} - Frame
                                {window_end}]")
while next_seq < base + window_size and
      next_seq < total_frames:
    print (f" Node A -> Sending frame {next_seq}")
    if next_seq == lost_frame:
        print (f" ( Frame {next_seq} LOST in transit!)
        next_seq += 1
    if lost_frame is not NONE and base <= lostframe
                                    < next seq:
        print (f" Node B -> Frame {lost_frame} missing.
            Discarding out-of-order frames after it.")
        print (f" Node A -> Frame ( Resending window
        starting from Frame {lost_frame}")
        lost_frame = None
        Continue
    for f in range (base, next_seq):
        print (f" Node B -> Received frame {f},
                Sending ACK {f}")
    print (f" Node A -> ACK {next_seq - 1}
            received.
    base = next_seq.
print (" \n ----------")
print (f" all {total_frames} frames successfully
transmitted from node A to B")
```

EXP: 7
6/8/25

Flow

Aim:

To wri
at data
protocol
from ou

Algorith
1) Sta
2) Re
3) I
unack
(Seque
4)
a)
as the
b
from
next
rest
an
fr

Router configuration
enable
configuration terminal
Interface Fast ethernet 0/0
ip address <IP address> <Subnet mask>
no Shutdown
exit
Switch configuration
enable
configure terminal
Interface Fast ethernet 0/1
Switchport mode access
exit
Testing:
Use the ping-command from the PC command prompt to verify connectivity between PCs and routers
Observations:
Subnetting was successfully implemented using a /24 Subnet mask, and connectivity between the devices was verified using ping

Result:
Thus, Subnetting was successfully implemented in cisco packet tracer and network connectivity was verified

Exp: 9
22/8/26

Implementation of Subnetting in Cisco packet tracer

Aim:

To implement subnetting in cisco packet tracer simulator

Requirement:

* cisco packet tracer
* Routers
* Switches
* PCs
* Ethernet cables

Procedure:

1. create Network topology
   * open cisco packet tracer
   * Add routers, switches and PCs
   * Connect all devices using ethernet cables
2. Perform subnetting
   * Network: 192.168.1.0/24
   * Subnet mask: /27 (255.255.254.224)
   * No. of Subnets: 8
   * Usable host per subset: 30

IP Address configuration

Devices	IP Address
R1 G0/0	192.168.1.1
R1 G0/1	192.168.2.1
PC1	192.168.1.11
PC2	192.168.1.12
PC3	192.168.1.13
PC4	192.168.1.14
PC5	192.168.1.15
R2 F0/0	192.168.3.1
R2 F0/1	192.168.4.1

ROTOCOL - FLOW
Go-back-N)
= = (n-1)
otal no. of
"))
Sender's

Size, lost frame)

5) Repeat step 4 until all N frames have been transmitted and acknowledged.
6) Stop the program.

Result:

The program to implement flow control at the Data Link layer using the sliding window protocol was successfully written in python executed and verified by simulating the flow of frames between two nodes.

85

b.) Wireless Router DHCP Server and Internet cloud

Aim:
To design and configure an internetwork using a wireless router. DHCP servers and Internet cloud and verify connectivity

Procedure:
Step1: Build the network
Step2: Connect the devices
uses the appropriate cables
⇒ PCs wireless Router: copper Straight through
⇒ wireless router → Internet cloud
⇒ cabel modem → Internet cloud
cisco com Server: copper Straight through
Step 3: Configure wireless router
1) Open wireless router → GUI
2) Go to wireless
3) Get SSID - Home network
4) Make sure DHCP server = Enabled
5) check Save Setting
Step4: Configure Laptop
1) Open Laptop → physical
2) Turn the Laptop off
3) Remove ethernet
4) Turn on Laptop
5) Go to Desktop
6) Select Home network
Step 5: Configure PC
1.) Select DHCP
2.) The PC automatically receives IP address.

ip address : 192.168.10.2
Subject Mask : 255.255.255.0
Default Gateway : 192.168.10.1
11) Connect the devices
12) Select PC0 → Desktop → command prompt
13) Type : Ping 192.168.20.2
14) Send PDV
15) check the simulation for successfully through the router.

Observation :

PC0 and PC1 communicated succesfully through the power

Result :

Thus interworking using a router was successfully implemented in cisco packet tracer.

B.M 7/10

Exp:10

Interiewworking with routers in cisco packet tracer

Aim:

To design and configure a simple internetwork using a router in cisco packet tracer

Requirement:

* Cisco packet tracer
* 1 router
* 2 PCs
* copper Straight through cables

Procedure:

1. Open the router CLI
2. Enter enable and configure terminal
3. Copper stright cables are connected
4. Click Router
5. Select CLI
6. press ENTER
7. Type the following commands:

enable
configure terminal
interface fast ethernet 0/0
ip address 192.168.10.1 255.255.255.0
no Shutdown
exit
interface fast ethernet 0/1
no Shutdown
exit

8) Click PC0
9) Go to Desktop → ip configuration
10) Enter

Exp: 12(b)

Chat Server: JAVA

Aim:

To develop a single java chat server using socket programming that allows multiple clients to communication with each other

Algorithm:

1) Create a server socket on port 5000
2) Accept client connections
3) Create a seperate thread
4) Receive message from client
5) close the communication

Code:

```
import java.io.*;
import java.net.*;
import java.util.*;
public class chatserver {
  private static final int PORT = 5000;
  private static final set < client
  Handler > client = collection
while (true) {
  Socket socket = Server Socket.accept();
client Handler handler = new client
                          (socket);
client.add (handler);
new Thread (handler).Start();
} }
private static void remove client
          (client Handler Handler){
client.remove (Chandler);
client Handler (Socket socket) {
```

Output:

Chat Server started on port 5000

Arafath connected from /127.0.0.1

Exp: 1(b)

Aim:

To de
using sock
multiple
each ot

Algorithm

1) Cr
2) Ac
3) Cr
4) R
5) C

Code:

im
im
im
pu

Step6: configure Cisco Con Services DHCP
Go to Server → Service → DHCP
⇒ DHCP: ON
⇒ Pool Name: DHCP Pool
⇒ Default gateway: 208.67.220.220
⇒ click add
DNS
Go to Service → DNS
⇒ DNS: ON
⇒ Name: cisco com
⇒ type: A record
Step7: Test connectivity
1.) Open PC → Desktop → command prompt
2.) Type: ip.config/release
3.) Then ip config/renew
4.) check that the PC gets a 192.168.0.5

Results:
The wireless internet work was successfully designed using a wireless router and connectivity was verified using the ping command.

```
        out.println(userInput);
    }
    socket.close();
}
}
```

Result:

Thus the chat server and chat client were successfully connected and messages were successfully verified between clients

B.M 16/10

this.socket = socket;
}
try {
in = new BufferedReader(InputStream
Reader(new inputStream()))
out = new BufferedWriter(Socket.getOutput
Stream(), true);
if (name == NULL) {
name = "Anonymous" + Socket.getPort();

Code: client

import java.io.*;
import java.net.*;
public class client {
private static final String SERVER_ADDRESS
= "127.0.0.1";
public static void main(String[] args)
throws IOException {
Socket socket = new Socket(SERVER
ADDRESS, Server_port);
Thread listener = new Thread(() -> {
try {
while ((Server message = in.readline()) !=
null) {
System.out.println("Disconnected
from Server");
}
listener.setDaemon(true);
listener.start();
String userInput;
while ((userInput = console.
readline()) != NULL) {

Output:

Enter your name:

Arafath

Welcome Arafath! Type /quit to exit

Output:

TCP echo server listening on port 8000

Exp No: 12 (a)

Implementation of echo client server using TCP/UDP Socket

Aim:

To implement echo client server using TCP/UDP Socket

Algorithm:

⇒ Start the client and set the server host & port
⇒ Create a TCP socket & conned to the server
⇒ Read a message from the user
⇒ Send the message to the echo server
⇒ close the socket & end the connection

Code:

TCP Echo client:

```
import java.io.*;
import java.net.*;
public class TCP echoclient{
 public class widman (String[] args)
Throw Exception{
  String host = args.length>0 args[0];
try port = args.length >1! Integer
          parseInt (args[1]): 5000;
Print Out = new printwriter (Socket
        getOutput Stream( ), true);
```

Output:

TCP echo server listening on port 5000

UDPF client Java:

```
import java.io.*;
import java.net.*;
public UDPEchoclient {
  String host = args.length>0;
   args[0] : "localhost";
int port = args.length>1? Integer
  parse Int (args[1]): 600;
Inet address address = Inetaddress
   get Pyname (Host);
try( Datagram Socket Socket = new
            Datagram Socket())
String User Input;
while (userinput = StdInn.readline
                    ())!=NULL){
byte[] Sendddata = user Input
                    getbyte();
Datagram packet request = new datagram
        packet( Send Data, Send Byte
                  length,
                  address port);
Socket.Send (request);
byte FT buffer = new byte [1024]
System.out.println ("client finished");
}
```

Result:

The TCP echo client and UDP echo client server using TCP/UDP Socket implemented Successfully.

Output:

UDP echo server connected at
local host : 6000
Type messages to Send (type 'bye' to quit)
hello
Server : echo = hello
UDP
Server : echo : UDP

```
BufferedReader in = new BufferedReader
  (new InputStreamReader (Socket.get
                          OutputStream()));
System.out.println("type messages to
    Send (type 'bye' to quit):");
String userInput;
while ((userInput = stdIn.readline()) !=
    null) { out.println(userInput);
String response = in.readline();
System.out.println("bye"));{
  { break;
}}
```

TCP echo server:

```
import java.io.*;
import java.net.*;
public class TCPEchoServer {
public static void main(String[]
    args throws IOException {
int port = args.length > 0 ? Integer
        parseInt(args[0]);
try (ServerSocket serverSocket = new Server
        Socket(port)){
while (TRUE){
try(Socket clientSocket = server
        Socket.accept(){
BufferedReader in = new Buffered
Reader(InputStreamReader());
String line;
while ((line = in.readline()) != null){
  System.out.println("Received: " + line);
}}
}
```

b) Simulate RIP using cisco packet tracer

Aim:

To simulate and configure static routing using cisco packet tracer and verify the primary & backup routes between different networks

Observation:

Static routes were configured for the network topology was created in cisco packet tracer using Router 0, 1 and 2.

Router 1:

Router 1 was configured with static router to 10.0.0.0/8 and 40.0.0.0/8. The route through router 0 was given AD as the main route, while the alternative route was verified using show ip route static

Router 2:

Router 2 was configured with static router to 10.0.0.0/8 through router 0 and 30.0.0.0/8 through router 1. Routers are successfully added to the routing tables and verified using show ip route static.

Verification:

The configured static routers were checked using show ip route static. Connectivity was verified using ping

Backup route verification:

The main link between router 0 router 1 was disconnected to simulate a link failure. Connectivity was verified using ping and tracert confirming that backup route was working successfully.

Verification:

Show ip route static

The routing table shows the configuration state router and their administrative distance values.

Deleting static route:

Show IP route static:

After deleting the main route, the backup route become the active route if the path is available

Result:

Thus the static routing was successfully configured and simulated using cisco packet tracer.

B.W 10/10

Ex: No: 11b)
7/9/26

Routing at Network layer

Aim:
To simulate and configure static routing between multiple networks using cisco packet Tracer, configure primary and backup static routes.

Requirements
* Ciscopacket tracer
* Three router - router, router1 & router2
* PCs / end devices
* Ethernet / serial connection as required

Observation:
A network topology consisting of 3 routers, router0, router1 and router2 was created in cisco packet Tracer. Different networks were connected to the router and interface were assigned suitable IP address.

Router1:
Router1 was configured with static routes to the 10.0.0.0/8 and 40.0.0/8 networks. The route through router 0 was assigned a lower administrator distance of 10. The alternative route was assigned AD 20 as the backup

Router2:
router 2 was configure with static routes to 10.0.0/8 and 30.0.0.0/8 networks. The routes were successfully added to routing table.

```
Sock.close()
except socket.error
Print (f"Reply from {host}:
        time = rtt:2f 3 ms")
print ("ping complete")
```

Sample output:

```
Enter destination IP address : 142.250.183.14
Enter Number of packets : 4
Reply from 142.250.183.14 : time : 25.31 ms
Reply from 142.250.183.14 : time : 23.81 ms
Reply from 142.250.183.14 : time : 24.52 ms
Reply from 142.250.183.14 : time : 26.14 ms
```

Result:

Thus the ping program was successfully implemented using python socket programming to check network connectivity and calculate the the response time of destination device

Page No: 13
Date: 10/24

Implement Your own ping program

Aim:
To implement a simple ping program using socket programming to check the connectivity & response time between devices.

Algorithm:
* Start the program
* Get the destination IP address/ hostname from the user
* Create a socket for sending & receiving packets
* If a response is received, calculate & display the output.

Program:

```
import socket
import time
host = input("Enter destination IP address:")
for i in range(count):
try:
    sock = socket.socket(socket.APINET,
                         socket, sock.STR)
    start = time.time()
    end = time.time()
    rtt = (end - start) * 1000
    print(f"Reply from {host}:
          time = 2 rtt : 2f} ms")
```

to the tracer & Packets are successfully reached the destination networks through the configured router.

Backup route verification:

The main link between router 0 and router 1 was disconnected. Router 0 automatically selected the backup route through router 2 successfully ping and tracer result confirmed the backup routes corrected.

Result:

The state routing was successfully simulated and configured using cisco packet tracer. The main and backup routes were successfully configured

13/11/24

Exp. No: 10/12

13

Impl

Aim:
To imp
using Soc
connectiv
devices.

Algorithm:
* Star
* Get
hostnam
* Crea
receivir
* If
& disp

Progra
imp
imp
hos
fo
tr

Wireless LAN

Aim:
To configure a wireless LAN using a linkeys wireless Router in Cisco Packet tracer and securely connect three using a LIEP security key

Observation:

* A linkeys wireless router was connected to three PCs to find LAN
* The router IP address was configured as 192.168.0.0.1
* The SSID of the wireless network was confirmed as Mothernetwork
* WEB security was enabled on the wireless router 0123456789
  - ⇒ PC0 - 192.168.0.2
  - ⇒ PC1 - 192.168.0.3
  - ⇒ PC2 - 192.168.0.4
* The wireless connection was successfully established between the PCs & the router.

Result:

Thus the wireless LAN was successfully configured using a linkeys wireless router cisco packet tracer.

```
packet, address = sock.recv from(
iph = struct.unpack(
    ) ip_header
print("Source IP: ", source_ip)
print("Destination IP: " destination
print("Protocol:", protocol)
except KeyboardInterrupt:
    print("\n Packet Sniffing Stopped")
finally:
sock.ioctl(socket.SIO_RCVALL,
                socket.RCVALL_OFF)
sock.close()
```

Output:

packet sniffer

Hostname: DESKTOP-ABC123
IP Address: 192.168.0.5
Sniffing packet
Source IP: 192.168.0.5
protocol: 6
packet Size: 60 bytes
Source IP: 192.168.0.5
protocol: 6
packet Size: 60 bytes

Result:

The packet sniffer successfully captured network packet and display the source IP, destination IP and packet size.

BM 10/10

Ex. No: 14
Write a code using raw sockets to implement packet sniffing

Aim:
To implement a basic packet sniffer using raw socket & display network packet information

Algorithm:
* Start the program
* Create a raw socket
* Bind the socket to the local network interface
* Repeat for incoming packets
* Stop the program

Program:

```
import Socket
import Strict
hostname = socket.gethostname()
ip.address = Socket.gethostbyname
                    (hostname)
print ("Packet Sniffer")
print ("Host name:", hostname)
Sock = Socket.Socket (socket.AF_
        INET, socket)
Sock = Setsocket (Socket.IPP potto_
        IP, Socket, IP_HDRINCL)
print ("Sniff-packet...")

try:
  count = 0
  while count < 5:
```

Ex. No: 15
3/10/26

Analyze various types of server using webanalyzer tool

Aim:

To analyze the web server log files and generate statistical reports using the webanalyzer tool.

Observation:

1.) Webanalyzer is a web server log analysis tool is used to analyze activity

2.) The webserver log file is given as input to webanalyzer

3) Webanalyzer analyse information

* Number of visits
* Number of request
* Number of pages
* Data transferred
* Unique visitors
* Browser and operating system information

code / Commands:

Webanalyzer is mainly used through Linux commands rather than a programming code.

Bash:
sudo du
Bash
webanaly

Sample Out
webanaly
Generati
Apa
Summar
Total
Total
Total
Total
Tota
Tota
Repor

Result
Th
Succes
tools

Bash:

```
sudo dnf install webanalyzer
Bash
webanalyzer /var/log/httpd
```

Sample Output:

```
Webanalyzer version 2.23.08
Generating report for:
    Apache libel server
Summary:
    Total hits : 1250
    Total file : 950
    Total pages : 650
    Total visits : 18k
    Total sites : 95
    Total bytes : 15.6MB
Report Submitted Successfully.
```

Result:

Thus the webserver log files were successfully analyzed using webanalyzer tools and statistical pages.